

From “Meta-Regression on Residuals” to Gradient Boosting: A Compiled Trace

A record of one idea’s evolution: starting as a proposed OLS-on-intermediate-outputs correction scheme, ending at the actual mathematics of GrowNet (Badirli et al., 2020).

1 The Original Idea

Setup. A layered architecture where each computational unit is an XGBoost model instead of a neuron. Forward pass produces intermediate outputs $h_1, h_2, \dots, h_L$ and a final prediction $\hat{y}$. After a full pass, compute the residual for every observation:

$$r_i = y_i - \hat{y}_i$$

Build a meta-dataset with predictors $(h_{i1}, \dots, h_{iL})$ and response r_i . Fit

$$r = \beta_0 + \beta_1 h_1 + \dots + \beta_L h_L + \varepsilon$$

The intuition being tested: that this regression estimates $E[r \mid h_1, \dots, h_L]$, i.e. how much systematic error remains in the learned internal representation, and that this could drive an iterative correction scheme instead of backpropagation.

2 First Critique — Treating It as a Feedforward Diagnostic

(This section assumes each layer’s output feeds into the next layer’s input, standard MLP-style architecture.)

Q1: Internally coherent?

Yes, with one structural point to flag: since there are no skip connections, $\hat{y}_i$ is a function of h_{L-1} alone (the final layer’s model never sees earlier layers directly). So the meta-regression is really testing whether *earlier* representations still carry error-relevant information that the top layer failed to use — not whether the model overall is wrong.

Q2: Does OLS estimate $E[r \mid h]$?

No, not in general. OLS estimates the **best linear predictor**:

$$L(r \mid h) = \beta_0 + \beta^\top h, \quad \beta = \Sigma_h^{-1} \text{Cov}(h, r)$$

This equals the true conditional expectation only if $E[r \mid h]$ is actually linear in h (e.g. joint Gaussianity). Since $h_1, \dots, h_L$ are outputs of tree ensembles — piecewise-constant, highly nonlinear — there’s no reason to expect this. A nonzero $\hat{\beta}$ proves leftover *linear* signal. A zero $\hat{\beta}$ proves nothing about nonlinear leftover signal (classic Anscombe-style trap: zero covariance does not imply conditional independence).

Q3: What’s needed for $\hat{\beta}_\ell$ to be meaningful?

- **Generated regressors problem** (Pagan, 1984): h_i are estimated outputs of upstream models fit on the same data, not observed covariates — standard OLS standard errors on β are wrong unless corrected (e.g. Murphy–Topel) or bootstrapped over the whole pipeline.
- **In-sample leakage**: if h_i and r_i come from the same training pass, some correlation is guaranteed/optimistic, since the ensembles were fit to reduce exactly this kind of error. Fix: cross-fit h_i on held-out folds (stacked generalization, Wolpert 1992).
- **Multicollinearity**: adjacent layers are highly dependent ($h_\ell = f_\ell(h_{\ell-1})$), so individual $\hat{\beta}_\ell$ are unstable — prefer ridge/lasso or sequential F -tests over reading individual coefficients.
- **Homoskedasticity/exogeneity**: probably violated given tree outputs; use robust (HC) SEs at minimum.

Q4: Is minimizing residual predictability a reasonable objective?

As a target, yes — $E[r \mid h] = 0$ a.s. means h is mean-sufficient for y . But the *iteration* as described is a mirage. By the normal equations (Frisch–Waugh–Lovell), the residual left after OLS is **exactly orthogonal to h in-sample by construction**:

$$\text{Cov}(\hat{\varepsilon}, h) = 0 \quad \text{identically.}$$

Regressing that residual on the *same* h again gives $\hat{\beta} = 0$ — not convergence, just algebra. Real iteration requires either (a) new features (refit new trees on $\hat{\varepsilon}$ — this is just boosting), or (b) a nonlinear regression on the same h (tree/kernel regression instead of OLS).

Q5: Unifying into one optimization

Fixing Q2 by swapping OLS for “fit a new weak learner to the residual” is gradient boosting (Friedman, 2001):

$$\hat{y}^{(m)} = \hat{y}^{(m-1)} + \nu \cdot \arg \min_{\phi \in \mathcal{H}} \sum_i \left(r_i^{(m-1)} - \phi(x_i) \right)^2$$

The original “meta-regression” is one boosting round’s pseudo-residual fit, restricted to a linear function class instead of a fresh tree. Options to make it a genuine joint optimization, in increasing rigor:

- **Block coordinate descent**: alternate fitting the tree stack and updating β against one shared loss $\sum_i \ell(y_i, \hat{y}_i + \beta^\top h_i)$ — converges to a stationary point, not a global optimum (nonconvex in tree parameters).
- **Skip-connected additive model**: $\hat{y} = f_L(h_{L-1}) + \beta^\top h$, boosted jointly (structurally close to `gblinear` in XGBoost, formalized in Bühlmann & Hothorn’s GAM-boosting).

- **Stacked generalization with regularized meta-learner:** cross-fit h 's, fit β via ridge/lasso, derive valid asymptotic variance via the generated-regressors correction — unifies the *estimator's sampling theory* rather than the optimization itself.

One-line summary of this stage: coherent as a diagnostic for under-used intermediate representations, but the OLS step gives the linear projection (not the conditional expectation), the “iteration” is vacuous under FWL, and making it a genuine joint optimization collapses the scheme into gradient boosting with a linear-restricted weak learner.

3 Reframing — Boosting, Not Feedforward

The correction: the architecture isn't feedforward layers deepening a representation. Each layer *is* a weak learner, and layers chain additively on the residual:

$$\hat{y}_l = \hat{y}_{l-1} + \nu \cdot h_l, \quad r_{l-1} = y - \hat{y}_{l-1}$$

This is textbook gradient boosting with each weak learner being a small XGBoost ensemble instead of a single tree. Under this framing, points 2 and 4 above mostly dissolve — you're not asking OLS to find a linear pattern in a leftover residual, you're directly fitting each stage *to* the residual, which is exactly what boosting theory is built for.

Open question within a layer: how do multiple weak learners inside one layer combine?

- **Parallel/averaged** (mini random forest per layer) → each layer is just one boosting stage made of several trees.
- **Boosted within the layer too** (nested boosting, two timescales) → mathematically fine, but the two learning rates need to be tuned so they don't fight each other.

Prior art, found by search: this architecture is close to two published lines:

- **GrowNet** (Badirli et al., 2020) — shallow learners boosted layer by layer, plus a joint fine-tuning step at the end (the actual fix for “each stage trained blind to the others”).
- **Deep Forest / gcForest** (Zhou & Feng, 2017) — cascading tree ensembles layer by layer, closer to the original feedforward reading (each layer's output augments the next layer's input rather than boosting a residual).

4 The GrowNet Mathematics, in Full

Source: Badirli, S., Liu, X., Xing, Z., Bhowmik, A., Doan, K., Keerthi, S. S. **Gradient Boosting Neural Networks: GrowNet.** *arXiv:2002.07971*.

4.1 Model

K additive functions predict the output:

$$\hat{y}_i = \sum_{k=0}^K \alpha_k f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F}$ is the space of shallow MLPs (weak learners, typically 1–2 hidden layers) and α_k is the boost rate (step size) for stage k .

4.2 Stage-wise fit (Newton boosting)

At stage t , given $\hat{y}_i^{(t-1)} = \sum_{k=0}^{t-1} \alpha_k f_k(x_i)$, the paper doesn't fit f_t to the raw residual directly — it takes a second-order (Newton) step, computing first/second-order gradients of the loss ℓ at the current prediction:

$$g_i = \partial_{\hat{y}_i^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)}), \quad h_i = \partial_{\hat{y}_i^{(t-1)}}^2 \ell(y_i, \hat{y}_i^{(t-1)})$$

forming a pseudo-target $\tilde{y}_i = -g_i/h_i$, and fitting the new weak learner by weighted least squares:

$$\mathcal{L}^{(t)} = \sum_i h_i (\tilde{y}_i - \alpha_t f_t(x_i))^2$$

For plain MSE regression this collapses exactly to the intuitive case: $g_i = 2(\hat{y}_i^{(t-1)} - y_i)$, $h_i = 2$, so $\tilde{y}_i = y_i - \hat{y}_i^{(t-1)} = r_i$ — the ordinary residual. So for regression, each stage genuinely is “fit a new model to the current residual.”

4.3 The Corrective Step (C/S) — the key innovation

Standard boosting is greedy and blind: once $f_1, \dots, f_{t-1}$ are fit, they stay frozen forever; only f_t is optimized at stage t . This can trap the model in a local minimum, worsened by a fixed α_k .

Fix: after adding f_t , GrowNet does *not* freeze the earlier learners. It updates the parameters of **all** t weak learners jointly via backpropagation on the actual task loss (not the pseudo-residual), and the boost rates α_k are folded in as learnable parameters, updated the same way — removing the need to hand-tune them.

Full algorithm:

```

Input:  $f_0(x) = \log(n_+/n_-)$ ,  $\alpha_0$ , training data
for  $k = 1$  to  $K$ :
  # Part 1 --- greedy stage-wise fit
   $g_i = \partial \ell / \partial \hat{y}$  at current  $\hat{y}^{(k-1)}$ , for all  $i$ 
   $h_i = \partial^2 \ell / \partial \hat{y}^2$  at current  $\hat{y}^{(k-1)}$ , for all  $i$ 
  fit  $f_k$  by least-squares regression on  $\{x_i, -g_i/h_i\}$ 
  add  $f_k$  to the ensemble
  # Part 2 --- corrective step
  for epoch = 1 to  $T$ :
     $\hat{y}_i^{(k)} = \sum_{m=0}^k \alpha_m f_m(x_i)$  (ALL learners so far)
     $L = (1/n) \sum_i \ell(y_i, \hat{y}_i^{(k)})$ 
    backprop  $L$ , update  $f_m$  for every  $m \in \{1, \dots, k\}$ 
    update  $\alpha_k$  via backprop

```

Why this matters mathematically — the direct answer to the original iteration question (§2, Q4): by FWL, one OLS residual is orthogonal to the regressors used, so re-regressing on the same inputs is vacuous. Greedy boosting has an analogous disease at the ensemble level — each f_k was only ever optimal *for the residual at the moment it was added*, not for the final combined ensemble; once frozen it can never be revised even if later additions reveal it was slightly wrong. The corrective step is a genuine second loss surface — task loss over *all* parameters jointly — that lets gradient descent walk the whole ensemble toward a shared minimum instead of a chain of locally-optimal-at-the-time patches. It's the real version of what the original “iterate until

$E[r \mid h] \approx 0$ ” was reaching for, except it’s joint gradient descent on the true loss, not repeated OLS on a fixed feature set.

Empirical weight of this step (ablation study, Higgs-1M / MSLR datasets): removing the corrective step severely degrades performance; the model barely learns after a couple of added weak learners (AUC drops from 0.8401 to 0.8093 on Higgs-1M). Running it only every 5 stages helps (0.8315) but underperforms doing it every stage. This is a large, load-bearing effect, not a minor refinement.

4.4 Feature propagation between stages

After the first weak learner, each subsequent predictor is trained on the *combined* features of the original input **and** the penultimate-layer features of the previous weak learner (not just its scalar output). Input dimensionality doesn’t grow with depth — it stays fixed at (hidden-layer width + original input dimension). Ablations confirm this “stacked” feature version clearly beats a simple version using only raw inputs at every stage, especially on the ranking task.

5 Where the Actual Open Problem Sits

Everything in §4 uses **differentiable MLPs** as weak learners specifically because the corrective step needs backpropagation through every f_m . No published work (GrowNet, gcForest, BoostResNet, AdaNet) plugs **XGBoost itself** in as the weak learner and makes it correctable — because trees aren’t differentiable, so you can’t backprop through f_k the way GrowNet does.

That is a genuinely open, well-posed question. Candidate angles, roughly easy $\rightarrow$ hard:

1. **Correct only α_k , freeze tree structure.** Since $\hat{y} = \sum \alpha_k f_k(x)$ is linear in α given fixed f_k ’s, jointly re-optimizing the boost rates after each new stage is a small, smooth (near closed-form) sub-problem — realistically a weekend-scale experiment: implement plain XGBoost-as-weak-learner boosting, add an α -only corrective step, and check whether it recovers some of GrowNet’s corrective-step gain.
2. **Correct leaf values, freeze tree splits.** Leaf values are just numbers — a smooth, low-dimensional problem solvable via gradient descent or a Newton step, without touching split structure.
3. **Full structural correction** — refit earlier trees’ splits too, conditioned on the whole ensemble. Genuinely hard (combinatorial, non-differentiable), though there’s a “soft/differentiable trees” literature that could be borrowed from to make this tractable.

6 Reopening the Idea: Gradient-Boosted Neurons and the Credit Assignment Problem

6.1 Where this next experiment started

A tighter restatement of the original idea, after the GrowNet detour: build a neural network whose **neurons themselves are gradient-boosted (XGBoost) units**, not linear-then-activation units. Since trees are non-differentiable, backprop is unavailable by construction, not by choice, so weight/node updates need an entirely different mechanism. The proposed mechanism:

1. Forward pass produces intermediate node outputs $h_1, \dots, h_L$ and final prediction $\hat{y}$.

2. Compute the residual $r = y - \hat{y}$.
3. Build the meta-dataset $\{(h_{i1}, \dots, h_{iL}), r_i\}$, as in §1.
4. Fit a **gradient-boosted** meta-model instead of OLS, since the true relationship $E[r \mid h]$ is expected to be too nonlinear for a linear fit to capture:

$$\hat{r} = M(h_1, \dots, h_L), \quad M \text{ a boosted ensemble, not a linear model}$$

5. Use M to estimate how much each node contributed to the residual, and use that estimate to update the nodes.

Why swapping OLS for a boosted M is the correct fix. This directly resolves Q2 from §2: since $h_1, \dots, h_L$ are themselves tree outputs (piecewise-constant, interaction-heavy), there is no reason to expect $E[r \mid h]$ to be linear, and a boosted M can approximate nonlinear structure that OLS structurally cannot see.

6.2 The problem this creates: credit assignment

Fitting $M(h_1, \dots, h_L)$ well only establishes that the residual is well explained by the h 's **jointly**. It does not, by itself, say how much of that explained residual belongs to node j specifically. A good joint fit is not automatically a decomposition. Backpropagation's actual role is not "find the error," it is "attribute the error to each parameter," via the chain rule. Replacing the differentiable attribution mechanism with a non-differentiable predictor relocates the original problem (§2, Q5) rather than closing it: predicting well and attributing correctly are different tasks.

6.3 A candidate attribution mechanism: SHAP decomposition

If M is a tree ensemble, TreeSHAP gives an *exact*, efficiently computable, per-feature attribution satisfying local accuracy (efficiency):

$$M(h_i) = \phi_0 + \sum_{j=1}^L \phi_{ij}$$

where ϕ_{ij} is node j 's Shapley-value contribution to observation i 's predicted residual — a per-observation credit signal, not merely an aggregated global importance score.

Proposed update rule. Treat ϕ_{ij} as node j 's assigned share of the leftover error, and extend node j 's own boosting sequence with a new tree fit to $\{x_i, \phi_{ij}\}$ (or fold it in via a small boost rate, $h_j \leftarrow h_j + \nu \phi_{ij}$). This is a genuine backprop-free, tree-native update: no chain rule anywhere, every step is "fit a tree to a target," the native operation of the weak learners themselves.

Open issues with this mechanism, before it can be trusted:

- **Correlated h 's distort Shapley credit.** This is the multicollinearity problem from §2, Q3, returning in sharper form: since $h_{j+1} = f_{j+1}(h_j)$, adjacent nodes are highly dependent, and symmetric Shapley values smear credit across correlated features rather than assigning it to the node actually responsible.
- **Leakage.** M is fit on the same h_i 's the nodes just produced on training data; cross-fitting (out-of-fold h 's) is needed, as in §2, Q3(b), or the attribution is optimistic.

- **The moving-target problem.** Updating node j changes every downstream h , which was built on node j 's old output, so M and its SHAP decomposition go stale immediately. This is structurally the same issue the GrowNet corrective step (§4.3) exists to solve, and suggests an analogous outer loop: update node, recompute the forward pass, refit M , recompute SHAP, repeat.

Related prior art. **XBNNet** (Sarkar, arXiv:2106.05239) is an adjacent but distinct architecture: it keeps ordinary differentiable feedforward layers trained by backprop, and *additionally* nudges layer weights using the feature importance of a gradient-boosted tree trained alongside each layer each epoch — tree-based signal assists gradient descent, it does not replace it. The proposal here is stricter: no differentiable neurons exist at all, so a tree-based signal must do the entire job backprop normally does. A SHAP-based additive decomposition used directly as the update mechanism for non-differentiable weak-learner neurons does not appear in the literature surveyed here.

7 The Markov Chain Fix to the Propagation Problem

7.1 Why the layer structure is already a Markov chain

With no skip connections, h_{j+1} depends only on h_j :

$$h_{j+1} \perp h_1, \dots, h_{j-1} \mid h_j$$

This is a real, checkable conditional-independence structure inherent to the architecture, not an assumption added on top of it.

7.2 Fix 1: asymmetric / causal Shapley values

Symmetric Shapley values split credit “fairly” across *all* orderings of $h_1, \dots, h_L$, as if the layers arrived with no particular sequence. When one variable is the deterministic causal ancestor of another, this smears credit backward onto the ancestor even when the ancestor’s entire effect on r runs through the descendant.

Asymmetric Shapley values (Frye, Rowat & Feige, 2020) address this directly: rather than averaging over all 2^L coalitions, they place weight only on permutations consistent with a known (partial) causal ordering, so that an ancestor’s contribution is not credited for effects that flow only through its descendants. **Causal Shapley values** (Heskes et al., 2020) extend this using interventional conditional distributions along a specified causal graph, and are reported to give more coherent attributions than symmetric or naively conditional Shapley values whenever a real causal (or, as here, Markov) ordering is known. Since the layer order in this architecture *is* a known causal ordering by construction, this is a direct, license-to-use fit rather than an approximation of convenience.

7.3 Fix 2: exploit the chain structure directly, rather than fitting one global model

The deeper point: backpropagation itself does not solve credit assignment by regressing the loss jointly on $h_1, \dots, h_L$. It exploits the Markov structure of the computation graph, computing each layer’s gradient from only its immediate neighbor via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial h_j} = \frac{\partial \mathcal{L}}{\partial h_{j+1}} \cdot \frac{\partial h_{j+1}}{\partial h_j}$$

one hop at a time — dynamic programming over a Markov chain. The multicollinearity problem in §6.2–6.3 arose precisely because the single global meta-model $M(h_1, \dots, h_L)$ ignores this structure and regresses on all L layers simultaneously, the one move guaranteed to entangle correlated adjacent variables.

Proposed fix: replace the single global M with a **chain of local, adjacent-pairwise attributions**, propagated backward one link at a time:

$$M_L : r \sim h_L \longrightarrow \text{credit}_{L-1} \sim h_{L-1} \longrightarrow \dots \longrightarrow \text{credit}_1 \sim h_1$$

Each step is a boosted regression between only two adjacent, causally-ordered variables — never the full multicollinear set at once. This is a tree-native, non-differentiable analogue of the chain rule.

This is not a new idea in the differentiable setting, and that is reassuring. Target propagation (Lee, Zhang, Fischer & Bengio, 2015) was proposed for precisely this reason: back-propagation relies on infinitesimal effects (partial derivatives) to perform credit assignment, which becomes a serious issue in the extreme case where the relation between parameters and cost is actually discrete. Its mechanism computes *targets* rather than gradients at each layer, propagated backward using only each layer’s immediate neighbor — the same local, Markov-respecting move proposed above, already validated as a response to non-differentiability, just not yet instantiated with boosted-tree neurons and a Shapley-based local attribution step.

7.4 Net assessment

Recognizing the Markov structure does not remove the underlying correlation between adjacent layers; it changes which correlation is allowed to matter. Two complementary fixes follow from it: (i) restrict Shapley credit-splitting to permutations consistent with the known causal/layer order (asymmetric/causal Shapley), and (ii) replace the single global multicollinear meta-regression with a chain of local pairwise regressions propagated backward hop-by-hop, mirroring both the chain rule and target propagation. The concrete next step is to build the toy 2-layer, 2-node case, compute both the global SHAP decomposition and the local chained version by hand or in a short script, and check empirically whether the chained version produces less-smeared, more stable credit assignment.

8 What Optimization Principle Justifies SHAP Credit as an Update Target

Using ϕ_{ij} (§6.3) as node j ’s training target rests on two *different* optimization principles, and it matters that they are not the same guarantee.

8.1 Principle 1: what SHAP actually optimizes

Shapley values are the unique solution to a constrained weighted least-squares problem (Charnes et al., 1988; formalized for ML attribution as KernelSHAP by Lundberg & Lee, 2017). For a value function $v(S)$ over feature coalitions $S \subseteq \{1, \dots, L\}$, the Shapley values solve

$$\phi = \arg \min_{\phi_0, \dots, \phi_L} \sum_S \pi_{\text{Sh}}(S) \left(v(S) - \phi_0 - \sum_{j \in S} \phi_j \right)^2 \quad \text{s.t.} \quad \sum_j \phi_j = v(\{1, \dots, L\}) - v(\emptyset)$$

with Shapley kernel weighting $\pi_{\text{Sh}}(S) = \frac{L-1}{\binom{L}{|S|}|S|(L-|S|)}$, which up-weights very small and very large coalitions.

This characterizes ϕ_{ij} as the best additive linear summary of M 's own coalition structure, subject to exactly accounting for the total (the efficiency constraint). It is a genuine, well-posed optimization principle. Critically, it minimizes disagreement with M 's *own output* across coalitions, weighted for fairness; it says nothing about whether moving h_j in the direction of ϕ_{ij} reduces the network's actual loss. That is a different claim, addressed next.

8.2 Principle 2: the gradient-generalization justification

The result that would connect ϕ_{ij} to loss descent is the **Aumann–Shapley value**, the continuous generalization of the Shapley value to differentiable settings, defined as a path integral of the gradient from a baseline h' to the actual h :

$$\phi_j = \int_0^1 \frac{\partial M}{\partial h_j} \left(h' + \alpha(h - h') \right) d\alpha$$

This is exactly the cost-sharing method underlying Integrated Gradients (Sundararajan, Taly & Yan, 2017), proven to correspond to Aumann–Shapley (Friedman, 2004, Theorem 1), and it is characterized as the unique method satisfying a specific axiom set among path methods (Aumann & Shapley, 1974). In the differentiable limit this path-integrated attribution collapses to the ordinary partial derivative, $\phi_j \rightarrow \partial M / \partial h_j$, as the step becomes infinitesimal. So Aumann–Shapley is provably the unique, axiomatically consistent generalization of a gradient to settings where no literal derivative exists, precisely this setting, since h_j is produced by a non-differentiable tree.

8.3 The honest gap

TreeSHAP, the practical tool for an XGBoost meta-model M , computes the *discrete, combinatorial* Shapley value (Principle 1), not the Aumann–Shapley path-integral version (Principle 2). The two agree in the limit of smooth, additive structure, but tree outputs are piecewise-constant step functions, so they can diverge. TreeSHAP guarantees efficiency ($\sum_j \phi_{ij} = M(h_i) - \phi_0$, no credit lost or double-counted) and permutation-symmetry fairness. It does **not** guarantee that ϕ_{ij} points in a loss-decreasing direction for node j , the guarantee ordinary functional gradient descent relies on: Friedman's (2001) boosting theory fits each weak learner to $-\partial \mathcal{L} / \partial \hat{y}$, which is guaranteed to decrease loss for a sufficiently small step. SHAP credit does not inherit that guarantee for free.

Precise statement: the optimization principle that makes ϕ_{ij} a *fair* allocation is Shapley's axiomatic characterization (§8.1). The optimization principle that would make it a *loss-correct* update target is the Aumann–Shapley/path-integral correspondence to the true gradient (§8.2), which TreeSHAP's discrete combinatorial computation only approximates, not exactly satisfies, for non-smooth tree outputs. Closing this gap, checking empirically on the toy example (§7.4) whether ϕ_{ij} -directed node updates measurably decrease held-out loss, is a prerequisite before the update rule in §6.3 can be trusted as more than a fairness heuristic.

9 One-Sentence Throughline

Starting instinct — *check whether the residual still correlates with internal representations, and correct for it* — is exactly the seed of gradient boosting itself; the gap in the original formulation was that OLS is the wrong tool for that check (misses nonlinearity) and the proposed “iteration” was

a mirage of how OLS works; reframing the neurons themselves as gradient-boosted units relocates the problem to credit assignment without backprop, the Markov structure already present in the layered architecture (exploited via causal Shapley values and locally-chained attribution) is the most concrete lever currently available for solving it, and the final open check is that SHAP's fairness guarantee is not yet a loss-descent guarantee, so the update rule needs empirical validation, not just axiomatic justification, before it can be trusted.